

Final Project Documentation

Jo Cooper

Southern New Hampshire University

CS 530: AI Principles and Applications

Professor Francis Akowuah

August 6th, 2026

Final Project Documentation

In this evaluation, I tested GitHub Copilot for its ability to handle code evaluation, bug correction, and code generation. In this assessment, I used Claude's Sonnet 5 model via Copilot. The assessment was aimed at determining the effectiveness of the tool and whether the implementation of its use in operations at TechNova Solutions is merited. To assess the efficacy of the tool, I performed three coding tasks. Firstly, I was given two code snippets containing logical and syntactical errors. I used the tool to assess and debug the snippets, and I did not perform manual correction. For the third task, I used the tool to generate a program that assesses the accuracy of two machine learning activation functions. The program generated included four functions. One function created a single neuron. The next defined a binary classifier function. Then, there is a function that creates a synthetic data set to be used by the other functions. The last function calculates weights for the generated data set and activation function. The program then outputs the weights, bias, and accuracy of both activation functions from a single generated data set. Assessment of these tests can be found later on in the paper.

Machine Learning and Deep Learning

Machine learning and deep learning are closely related topics. "Machine learning (ML) is a subset of artificial intelligence that focuses on the development of algorithms that allow computers to learn from and make decisions based on data." (zyBooks, 2026). Deep learning takes this a step further. "Deep learning is a specialized subset of machine learning ... It employs artificial neural networks, especially deep neural networks with multiple layers, to analyze various factors of data." (zyBooks, 2026). Deep learning models allow for more complex analysis of inputs, enabled by including potentially many hidden layers of neurons. Amini (2024) distinguishes these by separating machine learning as an AI that can learn without being

explicitly programmed, whereas deep learning expands on this to extract patterns from data with neural networks. Complex models such as ChatGPT and Claude will include many hidden layers of neurons. Given that deep learning is a subset of machine learning, this means that even complicated models use machine learning. That said, the more appropriate term would be deep learning given the depth of the neural networks used to generate outputs.

Natural Language Processing

Natural language processing is another subset of artificial intelligence. This field is "focused on using algorithms to understand, interpret, and generate human language and text." (zyBooks, 2026). An AI that uses NLP takes inputs and translates the words of the input into tokens. These tokens are then processed in a step called stemming that turns tokens into their base words. Similarly defined words are also changed into single base words in a process called lemmatization. Tokens are then tagged with their type of word, such as nouns and verbs. A model that utilizes NLP analyzes these tokens and outputs the most appropriate tokens based on the user's input. Additional steps might be included given the capabilities of a model. In the context of Copilot being used, how a token is tagged is going to function differently given that program code does not follow English sentence structure. Instead, the model will evaluate the code for syntactical errors in the same way a model can evaluate a sentence for grammatical errors. That said, for the same reason, logical errors can be difficult for a model to detect as coding logic is not consistent across systems. This can be seen in the second code snippet. The model can tell that many of the variables being assigned are not used. However, not all these variables were found on the first pass. Additionally, these models require context to perform their best. Telling the model to fix a specific bug might cause more issues if there are multiple ways the issue could be resolved. For instance, I clarified that unused code snippets should be removed from code

snippet two. Without this clarification, the model might determine that the most likely token output would include altering the output of the function to include these unused variables.

Differentiating Copilot from Other Tools

Copilot includes a few benefits over using other AI tools. Copilot allows the user to easily select which model they would like to use across various companies. Copilot also readily includes token and AI credit usage for the entire conversation and specific prompts. Keeping track of this allows users to mitigate token overuse, which is especially relevant with the growing costs of using these tools. These benefits can allow an engineer to choose an appropriate model while keeping token usage to a minimum. Yetiştiren et al (2023) tested the capabilities of Copilot in comparison to ChatGPT and Amazon CodeWhisperer. While the realm of AI models moves fast, their findings are noteworthy. They note where the training data for these models is sourced. ChatGPT is trained via "GitHub Repositories, OpenAI Codex Dataset, other code repositories". Amazon CodeWhisperer was trained with "Vast amounts of publicly available code". Lastly, Copilot is "trained on all languages that appear in public repositories". Due to the nature of being trained on publicly available data, all models can potentially struggle to understand proprietary code with specialized logic. Yetiştiren et al. (2023) additionally point out IDE support for each of the models. ChatGPT does not include IDE support. CodeWhisperer includes a small number of IDE's including Amazon specific ones. Copilot has support for many IDE's. When testing the performance of these models, they found ChatGPT was the most successful model at generating correct code, followed by Copilot and then CodeWhisperer. It is also found that new models of Copilot and CodeWhisperer demonstrated improvement in the rate of successful code generation. When comparing the security of the output code, Yetiştiren et al (2023) found "the generators are equally successful in terms of generating secure code."

Solving Real World Problems with Copilot

AI coding assistants can be used for various tasks in the coding workflow. Pandey et al (2024) discuss many tasks that software engineers used Copilot for in their workflows. These are separated into four categories. The categories are repetitive tasks, boilerplate, small function, and complex multi-file. Repetitive tasks are those that occur multiple times across various locations, such as documentation. Boilerplate includes code generation on widely used patterns. Small function describes custom code that is generated with relevant context in a small set of files. Complex multi-file are those that revolve around generated complex code across many files with dependencies. In their study, they found that uses for repetitive tasks saved the highest percentage of time and had the highest acceptance rate for the generated code. They also discuss the tool's reliability with various coding languages. They found it is most effective when used for Javascript, saving close to 50% of time. When it comes to debugging code, they note that the tool is most beneficial when the task is clearly described. This is expected as AI models tend to perform best when the input includes adequate context. Thoroughly describing the problem at hand is providing this context. They also specify that one of the key strengths of Copilot is that it "facilitates the onboarding process to new codebases by providing accurate code explanations." (Pandey et al, 2024).

Attributes and Limitations of Github Copilot

In the journal by Yetiştiren et al (2023), Github Copilot was found to generate correct code 46.3% of the time. It additionally showed an improvement rate of 18% for its new model at the time of testing, as mentioned before. While code security showed no meaningful difference between the tools, reliability and maintainability did show some differences. Copilot took the longest of the 3 models tested to eliminate errors in the code. Another limitation is if a company's

systems include specialized logic, the tool might not be able to understand this complexity properly. As shown with the code snippet corrections, while Copilot performs well at spotting and fixing syntactical errors, it does not excel at assessing even simple logic. "Copilot has limited utility for advanced proprietary code, such as code that implements unique business logic, or where the relevant code is distributed over many files." (Pandey et al, 2024). When utilizing Copilot for tasks such as this, it is important to frequently review the code to ensure the code performs properly for the given task. Pandey et al (2024) state that even though Copilot provides many benefits for the software development cycle, being aware of its limitations is important to maximize potential benefits.

Ethical Risks of AI-generated Code

Using AI code generation tools comes with a few intrinsic ethical concerns. These include environmental impact, system accountability, copyright, and workforce displacement. AI models take large amounts of electricity to run and use water to cool these systems. With growing public concerns surrounding the environmental impact of data centers and the models run off them, it is important to use AI responsibly and understand the environmental impact AI use has. System accountability refers to how due to AI automating the decision-making process, monitoring the system's accuracy and accounting for reliability is necessary (zyBooks, 2026). As noted above, these models are trained on publicly available data. However, there is no guarantee that this training data does not include code that was not intended for reuse. Lucchi (2024) states, "the vast majority of data that generative AI systems have assimilated – including both textual and visual content – have been de facto obtained without the express authorisation of the rights holders." When the models are trained on potentially unlicensed data, who owns the code that is output by the model? It falls to the developers to ensure that code output by a model does not

require additional licensing. This task is difficult however when the training data used by these models is not publicly available. The last ethical concern I will be discussing is that of workforce displacement. Artificial intelligence is a useful tool that can aid in the efficiency of many tasks, including coding and software development. However, this could result in humans losing their jobs, causing workforce displacement. While AI is a useful tool, it is important that AI does not infringe on people's ability to sustain themselves. "New job positions should also be developed to accommodate the workers; otherwise, it creates severe unemployment and social problems." (zyBooks, 2026). AI models could cause the need to perform specific tasks to be diminished. However, this could mean that jobs transition into roles where the task is to maintain the AI models and inspect outputs for errors. As AI models are prone to hallucinate in their outputs, maintaining and inspecting AI output will become a growing role in many industries.

Impact of Bias

All AI models include some amount of bias. This can come from the training data, developers, or users. When pulling together training data for a model, developers must work towards ensuring the data is diverse. "Algorithms used in AI and ML are often trained using historical data that may contain biases, so the resulting decisions can be unfair to certain groups." (Judijanto et al, 2025). Even if diverse training data is pulled, humans are inherently biased. Creating models based on human created data will then inherently include biases. When it comes to coding, these biases could cause the model to behave in certain ways. The publicly available content models are trained on will mean that the models will favor commonly accepted logic and algorithms. This will not necessarily be wrong for a given situation. On the other hand, this could mean that when a model is deciding the most likely output tokens, it will favor these common logic and code formats. As discussed earlier, this can create problems if the code being created

requires company specific logic to perform properly. When the model favors specific popular logic and algorithms, developers might find it necessary to repeatedly edit code manually so it fits their company's specific logic.

Privacy and Transparency

Privacy and transparency are major concerns when it comes to the use of AI models. As discussed earlier, when it comes to the transparency of how models are trained, it is unknown how models use inputs for the purposes of continued training. "Most of our surveyed developers said they do not trust AI-based programming assistants with sensitive data, and it is crucial to use them for this task very cautiously." (Madampe, 2025). The survey found that 64.71% of those surveyed distrust programming assistants when it comes to privacy. Al-Maamari (2026) tested multiple coding tools on their privacy capabilities. This test was based on 14 criteria broken into 3 categories: governance, technical, and control. Google Gemini scored the highest with an 89.25 out of 100. Copilot scored a 78.75, and OpenAI's GPT model scored a 68. They note that all models feature an opt-out for model training consent, however Claude instead was the only one to use an opt-in model. "Leaders like Gemini and Claude provide granular controls and clear retention policies, while others offer only global opt-outs and vague statements like keeping data 'for as long as is necessary.'" (Madampe, 2025). Because of these differences in privacy of input materials, the decision over which model is used becomes a security question.

Legal and Security Risks

There are a number of legal and security risks to consider. As discussed previously under the section of ethical concerns, one legal concern for using code generation tools is the possibility of generating code that is owned by a copyright holder. Using generated code without

accounting for this could result in legal action being taken by the copyright holder. System accountability is another ethical concern that can cause legal repercussions. When a model makes a decision that causes an error or breach, who is then responsible for this? This concern has come to a head recently with the news of OpenAI's model hacking another company's system during what was intended to be a closed space test. As noted by Reuters (2026), there are "questions about how closely AI companies are monitoring tests of increasingly powerful models, and could add fuel to calls for tighter oversight." In the aftermath of this incident, who is responsible for the actions of the model? Is OpenAI responsible as they are the owner of the model. Is it the developers that were monitoring the tests? Or, is it developers involved in the creation of the model? When testing code or models, developers must monitor the systems to prevent or stop security breaches like this from happening.

Github (2026) documentation outlines multiple risks and mitigation strategies for use of their coding tool. This documentation includes six specific risks. As with all code, using unvalidated code can introduce vulnerabilities to a system. While Copilot checks the code it generates for concerns prior to output, it is important for developers to manually validate code as well. The documentation also discusses how AI coding agents can push changes to repositories. To mitigate this, users can limit who can trigger the AI agent, where the agent can push to, and limit the agent's credentials. Other mitigation strategies include requiring human review for pushing updates and requiring additional pull request approval from someone other than the current user. Given Copilot's access to sensitive information, they also recommend limiting the model's access to the internet. AI models are also vulnerable when it comes to prompt injection. "To mitigate this risk, GitHub filters hidden characters before passing user input to Copilot cloud agent." (Github, 2026). The last risk discussed in the documentation is how automations run

without initiation. To mitigate this, automated tasks are attributed to the person that created the automation. Users can also control what tools are available for automated tasks. Additionally, automations from untrusted users cannot be triggered, and any automation that would require approval still does.

Long-Term Impact

The use of AI coding tools will inherently have an impact on the daily operations at TechNova Solutions. This is further discussed above in the section "Solving Real World Problems with Copilot." Developers using AI tools use the tools for code writing 82% of the time, while searching for answers and debugging are the next highest uses (Stack Overflow, 2024). Because of this heavy focus on generating code, the scope of employee responsibilities will change. "Companies are responding by refocusing staff on architecture, integration, and code review instead of simple feature-development work." (Belozerov et al, 2025). Belozerov et al (2025) further suggest that code includes a "provenance tag" that marks the model, prompt, and time a portion of code was created. Properly tagging generated code ensures that code will be manually reviewed before implementation. As the industry shifts towards implementing these tools in daily workflows, understanding prompting and manual review will become key skills. The provenance tagging mentioned would additionally become a valuable asset to implement as industry standard. Due to the nature of these changes, developers can take on many tasks. This potential autonomy can allow developers to create, document, debug, and test code more quickly and efficiently. While the need for second party review will still be important, the autonomy allowed by AI tools can aid in increasing the quality of code before moving the code to the next party.

Conclusion

Throughout this assessment, I discussed the capabilities, limitations, ethical implications, and impact of integrating GitHub Copilot into the software development workflow. Due to TechNova Solutions specialization in healthcare and financial industry solutions, the security capabilities of a tool should be of the utmost priority given the sensitive nature of the data. While Github Copilot includes security features, I would recommend the use of either Claude or Gemini given their granular control over what data the model retains. Additionally, Claude features an opt-in format instead of other tools' use of opt-out models. As discussed above, Copilot also took the longest out the 3 models tested by Yetiştiren et al (2023) at fixing bugs. Copilot does feature some benefits though. The tool easily integrates into IDE's and allows for users to select the AI model they would like to use. It additionally readily displays token usage metrics to allow developers to track token expenditure. Ultimately, the decision for what AI tool to implement in the workflow comes to two main factors, ease of implementation and security. Copilot can be easily implemented in IDE's and with simple token tracking, but additional processes would need to be implemented in the workflow to ensure the privacy of sensitive data is maintained. Claude on the other hand features granular security and is the only model with an opt-in model for data retention. However, token usage is not as readily available as it is with Copilot.

I additionally tested Copilot's capabilities at debugging and code generation tasks (GitHub Copilot, 2026). I used two code snippets with logical and syntactical errors. In the tests, the tool can easily find and fix syntax errors. However, the tool struggles to understand logical errors. An example of this can be seen in code snippet two. The tool could see that a while loop would not terminate because the variable is decremented instead of incremented. However, it does not recognize that the while loop is not performing a task for the overall function. Removal

of the unused variables also took multiple attempts, despite including notes to remove the variables. When generating code, the tool was able to quickly implement the functions as prompted without errors and with in-depth headers providing additional information. Running the program analyzes the training accuracy of sigmoid and step activation functions on a single synthetic data set. The output of the program shows that the sigmoid function showed an 84.5% training accuracy and the step function showed a 78.5% training accuracy. While the tool was successful in generating the desired functions, it is always important to review the code before further implementation as discussed above. From these tests, it can be seen the tool effectively identifies and fixes syntax errors and can generate correct code. For this purpose, Copilot can be effectively implemented at TechNova Solutions.

References

- Al-Maamari, A. (2026, May). Can You Trust Your Copilot? A Privacy Scorecard for AI Coding Assistants. In *Computer Sciences & Mathematics Forum* (Vol. 13, No. 1, p. 14). MDPI.
- Amini, Alexander. (2024, April 29). *MIT Introduction to Deep Learning | 6.S191*. YouTube.
https://www.youtube.com/watch?v=ErnWZxJovaM&list=PLtBw6njQRU-rwp5_7C0oIVt26ZgjG9NI
- Belozarov, V., Barclay, P. J., & Sami, A. (2025). *LLMs in Coding and their Impact on the Commercial Software Engineering Landscape*.
<https://doi.org/10.48550/arXiv.2506.16653>
- GitHub Copilot (2026, August 28) AI Principles Final [Generative AI Chat]. GitHub Copilot<https://github.com/copilot/share/c25451a6-01c4-8043-9102-cc4f84b160c2>
- GitHub. (2026). *Risks and mitigations for GitHub Copilot cloud agent*. GitHub Docs.
<https://docs.github.com/en/copilot/concepts/agents/cloud-agent/risks-and-mitigations>
- Lucchi, N. (2024). ChatGPT: A Case Study on Copyright Challenges for Generative Artificial Intelligence Systems. *European Journal of Risk Regulation*, 15(3), 602–624.
 doi:10.1017/err.2023.59
- Madampe, K., Grundy, J., & Arachchilage, N. (2025). How are we doing with using AI-based programming assistants for privacy-related code generation? The developers' experience. *arXiv preprint arXiv: 2503.03988*.

- Pandey, R., Singh, P., Wei, R., & Shankar, S. (2024). Transforming software development: Evaluating the efficiency and challenges of github copilot in real-world projects. *arXiv preprint arXiv:2406.17910*.
- Reuters. (2026, August 26). *OpenAI report says its network was hacked by its own rogue AI agents*. NBC News. <https://www.nbcnews.com/tech/tech-news/openai-report-says-network-was-hacked-rogue-ai-agents-rcna594590>
- Stack Overflow. (2024). *AI | 2024 Stack Overflow Developer Survey*. Stackoverflow.Co. <https://survey.stackoverflow.co/2024/ai>
- Yetiştiren, B., Özsoy, I., Ayerdem, M., & Tüzün, E. (2023). Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778*.
- zyBooks, a Wiley Brand. (2026) *CS 530 AI Principles and Applications*. <https://learn.zybooks.com/zybook/CS-530-10101.202657-1>